

Del código al objeto

Qué cambió la programación orientada a objetos respecto de lo que ya se sabe programar, y cómo eso se escribe en Python.

Rubén Schnettler • Docente

INACAP Valparaíso • Analista Programador

BLOQUE 1 DE 3

PRESENTACIÓN

Rubén Schnettler Lucero



linkedin.com/in/rubensch
rubenschnettler.cl



Formación

Ingeniero Civil Industrial · Magíster en Gestión Empresarial · UTFSM



Ejercicio profesional

Consultoría en gestión y desarrollo de software: modelamiento de procesos, indicadores y planificación estratégica



Docencia

Más de diez años · INACAP · U. Andrés Bello · U. de Las Américas

Laboratorio, no aula: al final de esta clase habrá código propio ejecutándose.

BLOQUE

01

El entorno de trabajo

Tres pasos, y queda listo para escribir

HÁGALO AHORA

Verificar Python y crear el entorno

Terminal

```
$ python --version
Python 3.13.2
$ cd Documentos/poo
$ python -m venv .venv
```

Si `python` no responde, probar con `python3` o `py`.

`$` No se escribe. Indica que la terminal **está esperando** un comando

`cd` *change directory* · entra a una carpeta

`-m venv` Ejecuta el módulo **venv**, que crea el entorno

`.venv` El nombre de la carpeta del entorno. El punto la deja oculta

Entorno virtual: una copia aislada de Python **para este proyecto solamente**. Lo que se instale ahí no afecta al resto del computador.

HÁGALO AHORA

Activar el entorno

Windows · PowerShell

```
> .venv\Scripts\Activate.ps1  
(.venv) >
```

macOS y Linux

```
$ source .venv/bin/activate  
(.venv) $
```



CÓMO SABER QUE QUEDÓ ACTIVO

Aparece **(.venv)** al principio de la línea. Si no aparece, **el entorno no está activado**.



SI WINDOWS LO BLOQUEA

PowerShell bloquea los scripts por omisión. Se habilita una sola vez con **Set-ExecutionPolicy - Scope CurrentUser RemoteSigned**, o se usa el símbolo del sistema con **activate.bat**.

PASO 3

El editor: Visual Studio Code

1 Abrir la carpeta, no un archivo suelto

File → **Open Folder**.

2 Instalar la extensión Python, de Microsoft

Aporta el resaltado, el autocompletado y el botón de ejecución.

3 Seleccionar el intérprete del entorno

Ctrl + **Shift** + **P** → **Python: Select Interpreter** → el que dice **.venv**.



EL PASO 3 ES EL QUE SE OLVIDA

Si el editor apunta al Python del sistema y no al del entorno, los errores se vuelven incomprensibles.



CÓMO VERIFICARLO

Abajo a la izquierda debe leerse **.venv**. Para ejecutar: el triángulo de arriba a la derecha, o **F5**.

BLOQUE

02

El cambio de paradigma

Qué introdujo la POO respecto del código que
ya se sabe escribir

EL PUNTO DE PARTIDA

El problema que la POO vino a resolver

Programación clásica • datos y funciones separados

Los datos, en variables sueltas

patente = "KXPR32"

km = 98200

La función, en otra parte

def sumar_km(actual, nuevos):

return actual + nuevos

km = sumar_km(km, 1900)



PROBLEMA 1 • NADA LOS UNE

Nada dice que esa patente y esos kilómetros son del mismo auto.



PROBLEMA 2 • NADIE PROTEGE EL DATO

Cualquier línea puede escribir **km = -500** y nada lo impide.



PROBLEMA 3 • SE REPITE

Si un camión es casi igual a un auto, hay que copiar y pegar.

LA RESPUESTA

Guardar juntos el dato y sus reglas

Programación orientada a objetos

```
class Auto:
    def __init__(self, patente, km):
        self.patente = patente
        self.km = km

    def sumar_km(self, nuevos):
        self.km += nuevos
```



VAN JUNTOS

La patente, los kilómetros y la operación que los modifica están **en el mismo lugar**.



LA CLASE PONE LAS REGLAS

Dentro de **sumar_km** se puede validar. Nadie modifica el dato por fuera.



Y SE PUEDE EXTENDER

Un camión **hereda** esto y agrega lo suyo, sin copiar nada.

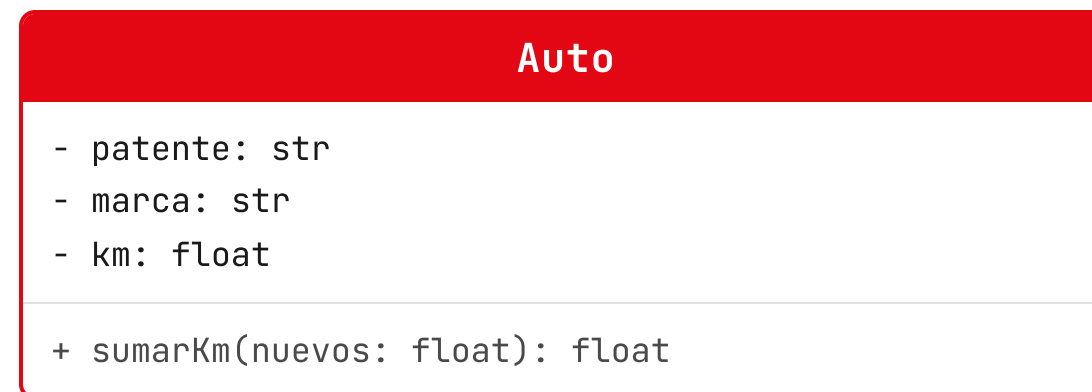
DOS FORMAS DE DECIR LO MISMO

El código y el esquema

Como código Python

```
class Auto:
    def __init__(self, patente, marca, km):
        self.patente = patente
        self.marca = marca
        self.km = km
    def sumar_km(self, nuevos):
        ...
```

Como esquema UML



UML • LENGUAJE UNIFICADO DE MODELADO

Una notación **estándar** para dibujar la estructura de un sistema: arriba el nombre, al medio los atributos con su tipo, abajo los métodos. **Dice lo mismo que el código**, pero se abarca de una mirada. Más adelante se trabaja en detalle.

POR QUÉ SE USA

Las cuatro ventajas concretas



Todo junto

Los datos de un auto y lo que se puede hacer con ellos viven en un solo lugar. Buscar un error toma minutos, no horas.



Se reutiliza

Lo común se escribe una vez y se hereda. Menos código es menos lugares donde equivocarse.



Datos protegidos

Se decide qué es visible y qué no. En una asignatura con «seguro» en el nombre, esto es el centro.



Crece sin romperse

Agregar un tipo de vehículo nuevo no obliga a tocar lo que ya funcionaba.

Ninguna de las cuatro se nota en un programa de veinte líneas. Todas se notan en uno de dos mil, que es el tamaño de cualquier sistema real.

BLOQUE

03

Los cuatro conceptos base

Clase, objeto, atributo y método

CONCEPTO 1 DE 4

La clase y el objeto

**CLASE**

La **definición** de un tipo de cosa: qué datos tiene y qué puede hacer. Se escribe una vez.

**OBJETO**

Un **caso concreto** de esa definición, con sus propios valores. Se crean tantos como hagan falta.

La clase: se escribe una vez

```
class Auto:
```

```
...
```

Los objetos: uno por auto

```
auto1 = Auto("KXPR32", "Toyota")
```

```
auto2 = Auto("HJLT18", "Suzuki")
```

Auto(...) El nombre de la clase con paréntesis **crea un objeto**. Se llama *instanciar*

auto1 Una variable que **apunta** a ese objeto. El nombre lo elige quien programa

EJERCICIO 1 • EN VOZ ALTA 5 minutos

¿Clase u objeto?

Determine, para cada uno de los seis casos, si lo nombrado es **una clase** o **un objeto**.

1 Perro

2 Rocky, el perro de la casa 3

3 Socio

4 La socia n.º 118, María Paz Rojas

5 Auto

6 El auto patente KXPR-32



PISTA

La pregunta que conviene hacerse es: **¿esto es el molde, o algo que salió del molde?**

RESPUESTA • EJERCICIO 1

Revisión en plenario

Clase o objeto: la respuesta

N.º	CASO	ES	POR QUÉ
1	Perro	Clase	Nombra un tipo de cosa, no a uno en particular
2	Rocky, el perro de la casa 3	Objeto	Se puede señalar con el dedo: tiene sus propios valores
3	Socio	Clase	Es la ficha en blanco con la que se registra a cualquiera
4	La socia n.º 118, María Paz Rojas	Objeto	Es una ficha llena: nombre propio y número propio
5	Auto	Clase	Vale para todos los autos del taller
6	El auto patente KXPR-32	Objeto	Uno solo, distinguible de los demás por su patente



LA REGLA

La clase **se escribe una sola vez** en el archivo. Los objetos **se crean tantas veces como haga falta**, y cada uno guarda sus propios valores.

11

CONCEPTO 2 DE 4

Atributos: los datos que guarda

```
self.patente = "KXPR32"  
self.marca = "Toyota"  
self.anio = 2019  
self.km = 98200.5  
self.activo = True
```

Cada dato tiene un **tipo**. En Python el tipo se deduce del valor; en el diagrama UML **se escribe siempre**.

str *string* · **texto**. Va entre comillas

int *integer* · **número entero**, sin decimales

float Número **con decimales**. El punto decimal «flota», de ahí el nombre

bool *boolean* · solo **True** o **False**, con mayúscula

date Una **fecha**. No viene incluida: se pide con `from datetime import date`

list Una **lista** de varios valores. Se escribe `[]`

CONCEPTO 3 DE 4

Métodos: lo que el objeto sabe hacer

```
def sumar_km(self, nuevos):  
    self.km += nuevos  
    return self.km
```



QUÉ HACE ESTE CÓDIGO

Le suma kilómetros al auto y devuelve el total nuevo. Si el auto tenía 98.200 y se le pasan 1.900, **el objeto queda con 100.100** y el método responde ese número.

Un método es una **función que vive dentro de una clase**. La diferencia con una función suelta es que recibe **self**.

def *define* · declara **una función o un método**

self **El objeto mismo.** Va siempre primero, y Python lo pasa solo: no se escribe al llamar

nuevos Un **parámetro**: el dato que el método necesita de afuera

+= Suma y guarda en un paso. `x += 5` equivale a `x = x + 5`

return **Devuelve** un valor a quien llamó, y termina el método

: Abre un bloque. Lo que sigue va **indentado**, y la indentación es obligatoria

EJERCICIO 2 • EN VOZ ALTA 5 minutos

¿Atributo o método?

Del enunciado de un taller y de un gimnasio salieron estas ocho frases. Indique cuáles serían **atributos** y cuáles **métodos**.

1 El color del auto

2 Arrancar el motor

3 Los kilómetros recorridos

4 Frenar

5 La patente

6 Registrar una visita al gimnasio

7 Si el socio está al día con el pago

8 La antigüedad del auto, en años

**PISTA**

Los **datos se guardan**; las **acciones se ejecutan**. Hay un caso que parece dato y no lo es.

RESPUESTA · EJERCICIO 2

Revisión en plenario

Atributo o método: la respuesta

1 **Atributo** · color, un dato que el objeto guarda

2 **Método** · arrancar, una acción que el objeto ejecuta

3 **Atributo** · km, un número que se guarda

4 **Método** · frenar, otra acción

5 **Atributo** · patente, y además identifica al objeto

6 **Método** · registrar_visita, cambia el estado del socio

7 **Atributo** · al_dia, un dato que vale True o False

8 **Método** · calcular_antigüedad. Ver la advertencia



EL CASO 8, QUE ES EL QUE SE FALLA

La antigüedad **no se guarda**: cambia todos los años y se puede deducir del año del auto. **Si un valor se puede calcular a partir de otro, es un método, no un atributo.** Guardarlo obligaría a corregirlo cada 1 de enero.

CONCEPTO 4 DE 4

El constructor: `__init__`

```
class Auto:
    def __init__(self, patente, marca):
        self.patente = patente
        self.marca = marca

# Al crear el objeto...
auto1 = Auto("KXPR32", "Toyota")
# ...Python ejecuta __init__ solo
```



SIN CONSTRUCTOR

El objeto **nace vacío**: habría que escribir a mano `auto1.patente = "KXPR32"` y cada atributo, sin que nada obligue a completarlos.



CON CONSTRUCTOR

Los datos **se exigen al crear**: si falta uno, Python avisa. Y todos los autos nacen con la misma estructura.

`__init__`

Se lee «init». Los dos guiones bajos marcan un **método especial**: Python ya sabe cuándo llamarlo. Con un solo guion no da error, pero el objeto nace vacío

CONVENCIÓN

Por qué fechaInicio en UML y fecha_inicio en Python



En el diagrama UML

`fechaInicio: date`

Se usa **camelCase**: primera palabra en minúscula y las siguientes con mayúscula inicial. Se llama así porque las mayúsculas parecen jorobas de camello.



En el código Python

`self.fecha_inicio`

Se usa **snake_case**: todo en minúscula, separado con guion bajo. Se llama así porque el texto se ve bajo y alargado, como una serpiente.



POR QUÉ SON DISTINTAS

UML no es de ningún lenguaje: usa la convención más difundida, heredada de Java. Python tiene **su propia guía de estilo**, la PEP 8, que exige snake_case. **Ninguna está mal: cada una manda en su territorio.**

Las clases sí coinciden: **Auto** con mayúscula inicial en los dos lados. Eso se llama PascalCase.

EL ERROR MÁS FRECUENTE

Cuándo algo es una clase y cuándo es un atributo



MODELADO INCORRECTO

Crear una clase **Patente**, otra **Marca** y otra **Kilometraje**, cada una con un solo dato dentro.

Prueba 1 ¿Existe por sí solo? Un auto, sí. Una patente suelta no significa nada

Prueba 3 ¿Hace algo? Si no tiene ni un método razonable, es un atributo



MODELADO CORRECTO

Los tres son **atributos de Auto**. Ninguno existe sin un auto que los tenga.

Prueba 2 ¿Tiene varios datos propios? Un cliente tiene nombre, teléfono y correo: es clase

Prueba 4 Al borrar el objeto dueño, ¿el dato pierde sentido? Entonces era un atributo

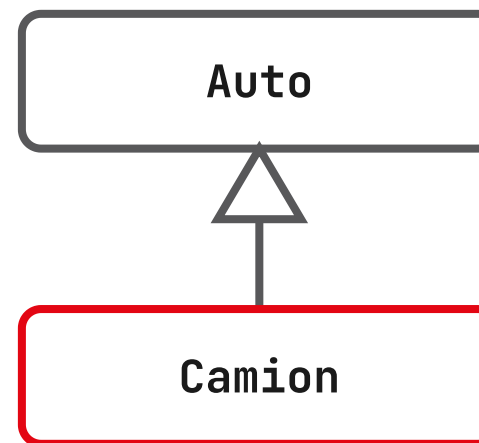
EL CUARTO PILAR EN CÓDIGO

Herencia: escribir una vez lo que se repite

```
class Camion(Auto):  
    def __init__(self, patente, marca, carga):  
        super().__init__(patente, marca)  
        self.carga = carga
```

Un camión **es un** auto: ya tiene patente, marca y el método `sumar_km`, sin repetir una línea.

Lo mismo, en UML



(Auto) El paréntesis indica **de quién hereda**. En UML es **el triángulo vacío**, que apunta siempre a la clase padre

super() Ejecuta el constructor del padre para no repetirlo

EJERCICIO 3 • EN PAREJA 6 minutos

La clase padre que falta

Un sistema de flota registra tres tipos de vehículo. Estos son sus atributos:

Auto

- patente: str
- marca: str
- km: float
- nroPuertas: int

Camion

- patente: str
- marca: str
- km: float
- capacidadCarga: float

Moto

- patente: str
- marca: str
- km: float
- cilindrada: int

**RESPONDA TRES COSAS**

1. ¿Qué atributos se repiten en las tres? · **2.** ¿Cómo llamaría a la clase que los reúne? · **3.** ¿Qué atributo queda en cada una de las tres?

RESPUESTA · EJERCICIO 3

Revisión en plenario

Lo repetido sube; lo propio se queda

Vehículo

- patente: str
- marca: str
- km: float

▲ heredan de ella

Auto

- nroPuertas: int

Camion

- capacidadCarga: float

Moto

- cilindrada: int

```
class Vehiculo:
    # patente, marca, km
    ...

class Camion(Vehiculo):
    # solo lo suyo
    ...
```



LA REGLA

Patente, marca y kilómetros se escriben **una sola vez**. Corregir un error ahí lo corrige en las tres.

BLOQUE

04

A escribir código

Primero se modela en pantalla, después lo
escribe cada uno

LO HAGO YO • EL ENUNCIADO

Un taller de autos

**ENUNCIADO**

Un taller registra los autos que atiende. De cada auto interesa la **patente**, la **marca** y los **kilómetros**. El taller necesita poder **sumar los kilómetros** de cada atención, y **preguntar si al auto le toca mantención**: a partir de los cien mil kilómetros.

**Las tres preguntas**

- ¿Cuál es la clase?
- ¿Cuáles son los atributos, y de qué tipo?
- ¿Cuáles son los métodos?

Se responden en ese orden, y de ahí sale el código.

LO HAGO YO • PASO 1

Del enunciado a la clase y sus atributos

	EN EL ENUNCIADO	QUÉ ES	TIPO	POR QUÉ
A	los autos que atiende	La clase	—	Existe por sí mismo y tiene varios datos
B	patente	Atributo	str	Es texto: mezcla letras y números
C	marca	Atributo	str	Texto. No es clase: aquí no tiene datos propios
D	kilómetros	Atributo	float	Puede tener decimales
E	sumar los kilómetros	Método	—	Es un verbo: algo que el objeto hace
F	preguntar si le toca mantención	Método	—	Otro verbo. Responde sí o no: devuelve bool

Los sustantivos son candidatos a clase o atributo; los verbos son métodos. Las letras A a F reaparecen en el código de las próximas diapositivas, para ver dónde quedó cada decisión.

SE ESCRIBE EN PANTALLA

El esqueleto de la clase, línea por línea

taller.py

```
A class Auto:
    def __init__(self, patente, marca, km):
        B self.patente = patente
        C self.marca = marca
        D self.km = km
```

- 1 `class` es la palabra con la que Python **declara una clase**. El nombre va en singular y con mayúscula inicial, y la línea **termina en dos puntos**
- 2 Todo lo **corrido cuatro espacios** pertenece a la clase. Python usa la sangría donde otros lenguajes usan llaves
- 3 `__init__` es el **constructor**: Python lo ejecuta solo cada vez que se crea un auto
- 4 `self` es **el objeto que se está creando**. Va siempre como primer parámetro, y Python lo pasa solo
- 5 `patente, marca, km` son los **parámetros**: los tres datos que hay que entregar al crear el auto
- 6 A la izquierda del igual, `self.patente` es **el atributo que queda guardado**; a la derecha, el parámetro que llegó. Pueden llamarse igual sin confundirse

SE ESCRIBE EN PANTALLA

Los dos métodos, línea por línea

taller.py • dentro de la clase

```
E      def sumar_km(self, nuevos):  
        self.km += nuevos  
        return self.km  
  
F      def necesita_mantenccion(self):  
        return self.km ≥ 100000
```

- 7 Los métodos van **al mismo nivel que `__init__`**: cuatro espacios. Si quedan fuera, dejan de pertenecer a la clase
- 8 `nuevos` es lo que el método **recibe de afuera**: los kilómetros de esta atención
- 9 `self.km += nuevos` lee el valor actual, le suma y lo vuelve a guardar. Equivale a `self.km = self.km + nuevos`
- 10 `return` **devuelve** el kilometraje nuevo a quien llamó al método, y termina ahí
- 11 El segundo método **no recibe nada**: solo mira su propio dato. Por eso lleva únicamente `self`
- 12 La comparación `≥` ya produce `True` o `False`. **No hace falta un `if`**

LO HAGO YO • PASO 4

El archivo completo, hasta aquí

taller.py

```
class Auto:

    def __init__(self, patente, marca, km):
        self.patente = patente
        self.marca = marca
        self.km = km

    def sumar_km(self, nuevos):
        self.km += nuevos
        return self.km

    def necesita_mantenccion(self):
        return self.km ≥ 100000
```



ESTO ES TODO EL ARCHIVO

Doce líneas. Las tres partes que se explicaron por separado, ahora juntas: la declaración, el constructor y los dos métodos.



SI SE EJECUTA AHORA

No pasa nada, y está bien. Una clase es solo una definición: falta crear un objeto y pedirle algo.

SE ESCRIBE EN PANTALLA

Crear un auto y preguntarle algo

taller.py • el archivo completo

```
class Auto:
    def __init__(self, patente, marca, km):
        ...
    def sumar_km(self, nuevos):
        ...
    def necesita_mantenion(self):
        ...

auto1 = Auto("KXPR32", "Toyota", 98200)
print(auto1.marca)
print(auto1.km)
auto1.sumar_km(1900)
print(auto1.necesita_mantenion())
```

- 13 Lo **resaltado es lo nuevo**, y va en el mismo archivo, debajo de la clase. Los puntos suspensivos solo indican que el código de arriba sigue igual
- 14 Estas líneas van **pegadas al margen**: ya no están dentro de la clase, sino en el programa que la usa
- 15 Al escribir `Auto(...)` se ejecuta `__init__` con esos tres valores, y **nace el objeto**
- 16 El punto en `auto1.marca` **entra al objeto** y lee un atributo
- 17 `sumar_km(1900)` **cambia el estado**: el auto queda con 100.100. Nótese que **no se escribe self**
- 18 Los paréntesis en `necesita_mantenion()` lo distinguen de un atributo: **es un método, se ejecuta y responde**

SE EJECUTA EN PANTALLA

Ejecutar y leer la salida

Terminal

```
$ python taller.py  
Toyota  
98200  
¿Necesita mantención? True
```

- 19 `Toyota` y `98200` son los dos atributos, tal como se guardaron al crear el objeto
- 20 La tercera línea es **una consulta**: se le preguntó al objeto y respondió `True`, porque $98.200 + 1.900$ pasó de los cien mil
- 21 El objeto **recordó** el cambio entre una línea y la siguiente. Eso es lo que una variable suelta no hace

SE ESCRIBE EN PANTALLA

La consulta que sí se parece a un sistema

taller.py • el archivo completo

```
class Auto:
    # ... la clase, igual que antes

autos = [
    Auto("KXPR32", "Toyota", 100100),
    Auto("HJLT18", "Suzuki", 41500),
    Auto("BGDR54", "Kia", 132000),
]

print("Autos que necesitan mantención:")
for a in autos:
    if a.necesita_mantenecion():
        print(" -", a.patente, a.marca)
```

- 22 **Lo resaltado es lo nuevo:** reemplaza al auto suelto del paso anterior
- 23 Los corchetes `[ ]` crean una **lista**: tres objetos juntos
- 24 `for a in autos:` recorre la lista **uno por uno**. En cada vuelta, `a` es el auto de turno
- 25 `if` ejecuta la línea siguiente **solo si** la respuesta es `True`
- 26 **Doble sangría:** el `print` está dentro del `if`, que está dentro del `for`

SE EJECUTA EN PANTALLA

El resultado de la consulta

Terminal

```
$ python taller.py
```

```
Autos que necesitan mantención:
```

- KXPR32 Toyota
- BGDR54 Kia



ESTO YA ES UN SISTEMA PEQUEÑO

Se guardaron tres autos, se les hizo **la misma pregunta a todos**, y solo respondieron los que correspondía. El Suzuki, con 41.500 kilómetros, quedó fuera solo.



LO QUE HAY QUE RETENER

La regla de los cien mil kilómetros está escrita **en un solo lugar**: dentro de la clase. Si el taller la cambia a ochenta mil, se corrige una línea y los tres autos responden distinto.

AHORA USTED · ACTIVIDAD INDIVIDUAL

35 minutos

Escribir, ejecutar y consultar



ENUNCIADO

Un gimnasio registra a sus socios. De cada socio interesa el **nombre**, el **número de socio** y si está **al día con el pago**. El gimnasio necesita poder **registrar una visita** y **preguntar si el socio puede entrar**: solo si está al día.

Es la misma estructura del taller: cambia el dominio, no el método. **Las visitas empiezan en cero**, así que no van entre los parámetros.



Y DESPUÉS, CON INTELIGENCIA ARTIFICIAL

Cuando el código esté **escrito y ejecutando**, pedirle a una herramienta de IA que lo revise y proponga mejoras. Anotar **qué sugirió y qué se aceptó o rechazó**.



Qué entregar

- La clase, con sus atributos
- Los dos métodos
- Tres socios en una lista
- La consulta y su salida
- Lo que sugirió la IA, y qué se hizo



LOS TRES ERRORES DEL PRIMER DÍA

Olvidar **self** · escribir **_init_** con un guion · perder la sangría.

La clase se escribe una vez; los objetos, tantos como haga falta

La actividad queda para trabajar en casa. La próxima sesión parte revisándola.

TI3V21 • Programación Orientada a Objeto Seguro • INACAP Valparaíso